PART IV — FROM RAG TO AGENTIC RAG

Chapter 22B

Semantic Compression of Retrieved Context

"Retrieval finds evidence. Compression decides how much of it the model actually needs to read."

Chapter 22B — Semantic Compression of Retrieved Context

A retriever tuned the way Chapters 11 and 12 describe returns real, relevant, well-scored chunks — and still hands the model far more redundancy than it needs. Overlapping chunk boundaries repeat the same sentence twice. Multiple source documents state the same fact in different words. A chunk that scored 0.8 on the whole is often only one sentence relevant to this specific question, with the rest along for the ride.

This chapter builds the answer: a three-stage compression pipeline — Neighbor-Aware Compression, Deduplication Compression, and LLM-Based Compression — each one narrower and more expensive than the last, each one guarded by its own LLM-as-judge validator so that compression can never quietly delete a fact it was supposed to preserve. Every mechanism here is real, shipped code from `context_compression.py` and `validators.py`, including the guard conditions that exist because an earlier version got exactly one of these steps wrong.

By the end of this chapter you will be able to build a layered compression pipeline that merges, deduplicates, and query-filters retrieved context in that order, pair every compression stage with a validator that can reject its own output, and recognize the specific structural bugs — intra-chunk false positives, greedy-regex extraction failures, unchecked over-compression — that a compression pipeline like this one will eventually produce if left unguarded.

22B.1 Why retrieved chunks are redundant by design

> **Definition — Context redundancy**
> Retrieved content that repeats information already present elsewhere in the same context window — across chunk boundaries, across source documents, or within a single chunk relative to the specific question being asked — costing prompt tokens without adding evidence.

Redundancy is not a retrieval bug; it is a direct, intended consequence of decisions made two chapters ago for good reasons. Chapter 7's `chunk_overlap=200` exists specifically so that a fact sitting near a chunk boundary is never lost to a single unlucky split — and the same overlap that prevents loss guarantees that consecutive chunks share real, repeated text. Multiple source documents describing the same underlying fact — Chapter 14.2's merge-prompt example shows two different files both stating "1 in 36 children" almost verbatim — compound the effect further, entirely independent of chunking.

None of this is a reason to chunk differently or retrieve less. It is a reason to add a stage between retrieval and generation whose only job is deciding what the model actually needs to read, now that the evidence has already been found.

22B.2 The three-stage compression hierarchy

NAC, DC, and LBC run in a fixed order, each addressing a narrower and more expensive question than the one before it — and each stage's output is exactly what the next stage's input assumes it will receive.

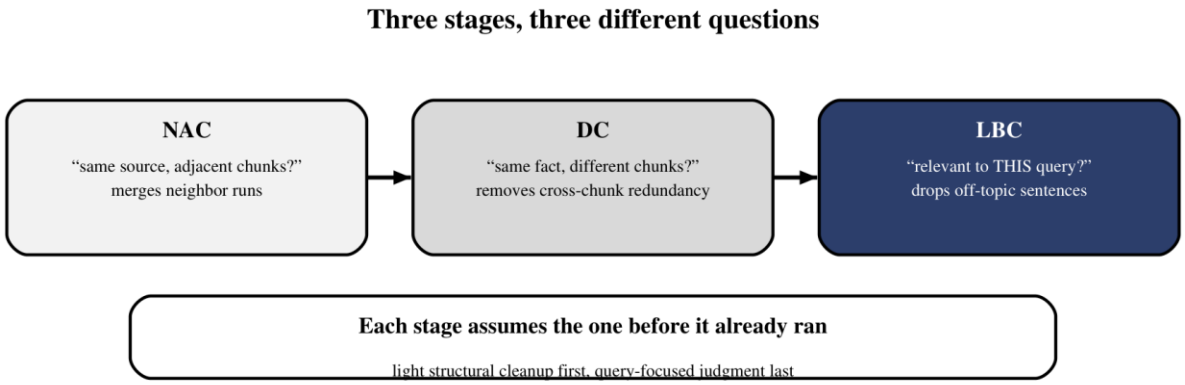

Figure 22B.1 — Structural cleanup runs before semantic judgment; cheap, mechanical fixes come first.

```
def compress_context_pipeline(chunks, query, llm, embedding_manager,
judge_llm):
    if ENABLE_NAC_COMPRESSION:
        chunks = compress_neighbor_chunks(chunks, llm, embedding_manager,
judge_llm)
    if ENABLE_DC_COMPRESSION:
        chunks = deduplicate_compression(chunks, llm, judge_llm)
    if ENABLE_LBC_COMPRESSION:
        chunks = llm_based_compression(chunks, query, llm, judge_llm)
    return chunks
```

The ordering in Figure 22B.1 is deliberate, not arbitrary. NAC runs first because it needs no query at all — it only needs `chunk_seq` metadata, so it can restore document flow before anything else touches the content. DC runs second because deduplication across a merged, flow-restored set of chunks finds cleaner redundancy than deduplication across raw, boundary-split fragments would. LBC runs last, and is the only stage that reads the query, because query-focused filtering only makes sense once the chunks it is filtering already represent the corpus's own best, least-redundant statement of the facts.

22B.3 Neighbor-Aware Compression (NAC)

> **Definition — Neighbor-Aware Compression**
> Merging consecutive chunks from the same source document — identified by adjacent `chunk_seq` values — into one consolidated chunk before any other compression runs, restoring continuity that fixed-size chunking split apart.

NAC's premise is narrow and mechanical: two chunks from the same file, with consecutive sequence numbers, are very likely two halves of one continuous passage that chunking happened to cut. Merging them is safe precisely because it needs no judgment about content — only two facts already sitting in the metadata.

22B.3.1 Detecting neighbor runs via chunk_seq metadata

Chunks are split into those carrying an integer `chunk_seq` and those that don't — a learned-QA chunk, for instance, has no sequence position to merge by — and only the eligible set is scanned. Sorted by source and sequence, a run is any maximal stretch where each chunk's sequence number is exactly one more than the last.

```
eligible.sort(key=lambda x: (x[1]["source"], x[1]["chunk_seq"]))
runs, current_run = [], [eligible[0]]
for orig_idx, chunk in eligible[1:]:
    prev = current_run[-1][1]
    if chunk["source"] == prev["source"] and chunk["chunk_seq"] ==
prev["chunk_seq"] + 1:
        current_run.append((orig_idx, chunk))
    else:
        runs.append(current_run)
        current_run = [(orig_idx, chunk)]
runs.append(current_run)
```

A run of length one is a singleton and passes through untouched — NAC never forces a merge where there is no neighbor to merge with.

22B.3.2 The NAC merge prompt and validate-merge loop

Every run longer than one chunk is handed to the same `_CHUNK_MERGE_PROMPT` Chapter 14.2 already introduced — consolidate, preserve every fact, cite every source inline — and its output is checked by `validate_merge` (Section 22B.6.2) before being accepted.

```
candidate = _merge_similar_chunks(source_chunks_for_merge, llm,
embedding_manager, feedback=feedback)
check = validate_merge(source_chunks=source_chunks_for_merge,
merged_chunk=candidate, judge_llm=judge_llm)
if check["verdict"] == "FAITHFUL":
    merged = candidate   # accepted
```

22B.3.3 Retry-with-feedback when the merge is unfaithful

An `UNFAITHFUL` verdict does not discard the run — it feeds the judge's own fabricated- and dropped-claim lists back into the prompt as explicit correction instructions, and retries up to `LLM_RESPONSE_RETRY_LIMIT` times before giving up.

```
issues = [f"- FABRICATED: \"{c}\"" for c in check["fabricated_claims"]]
issues += [f"- DROPPED: \"{c}\"" for c in check["dropped_claims"]]
feedback = "\n".join(issues) or check["overall_reason"]
```

If every attempt is rejected, NAC does not force a bad merge through — it abandons the merge entirely and keeps the run's original chunks unmerged. A compression stage that can only make things smaller, never wrong, has an easy fallback: do nothing.

22B.4 Deduplication Compression (DC)

> **Definition — Deduplication Compression**
> Scanning a sliding window of chunks for sentences in different chunks that state the same fact, confirming genuine redundancy with a judge, and removing all but one instance of each confirmed duplicate.

Where NAC merges whole chunks by metadata alone, DC works at sentence granularity and requires actual semantic judgment — two sentences can share a topic without sharing a fact, and only the second is safe to remove.

22B.4.1 The sliding-window scanner and the DC scan prompt

Chunks are scanned in fixed-size windows (`DC_WINDOW_SIZE = 3`) rather than all at once — a window keeps the scan prompt small and lets redundancy detection scale to large chunk counts without a single, ever-growing context.

```
for window_start in range(0, len(result), window_size):
    window = result[window_start : window_start + window_size]
    prompt = _DC_SCAN_PROMPT.format(chunks_block=chunks_block)
    flagged = llm_invoke(llm, [...], caller_tag="DC")       # proposed
redundancy groups
```

`_DC_SCAN_PROMPT` (Chapter 14.2) is deliberately paranoid about what counts as redundant — same subject, same claim, same meaning, with worked GOOD and BAD examples distinguishing genuine duplication from merely-related content — because this scan is a proposal stage, not a final decision.

22B.4.2 The redundancy judge and validate_redundancy

Every group the scanner proposes is re-checked by `validate_redundancy` (Section 22B.6.3) before anything is removed — the scanner and the judge are deliberately two separate LLM calls with two separate prompts, so a scanner's overeager proposal has an independent check to survive.

22B.4.3 The intra-chunk group bug and the cross-chunk guard

An early version of the scanner occasionally proposed a "redundancy" group whose members were two sentences from the *same* chunk — which is not cross-chunk redundancy at all, just a chunk repeating itself, and removing one instance would corrupt that chunk's own content for no real gain.

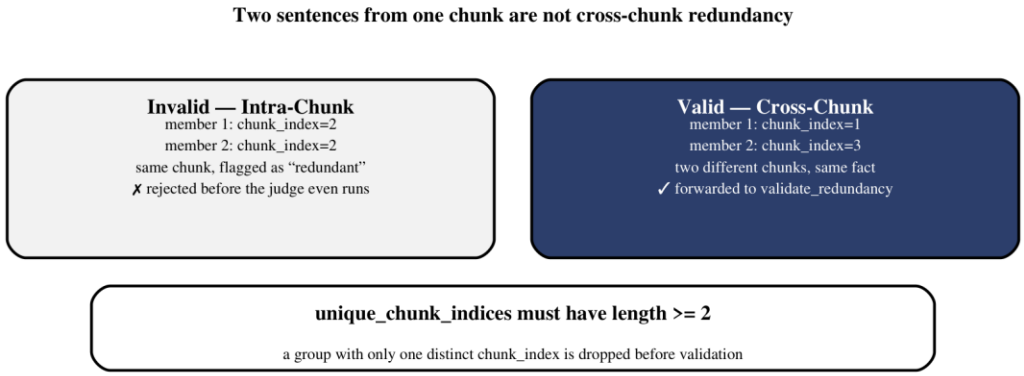

Figure 22B.2 — A group needs members from at least two distinct chunks to count as redundancy worth removing.

```
unique_chunk_indices = {m["chunk_index"] for m in clean_members}
if len(unique_chunk_indices) < 2:
    logger.debug("group dropped — intra-chunk repetition, not cross-chunk
redundancy")
    continue
```

The guard in Figure 22B.2 runs before the group ever reaches `validate_redundancy` — an intra-chunk group is structurally invalid regardless of what a judge would say about it, so there is no reason to spend a judge call finding that out.

22B.4.4 Bracket-counting JSON extraction

The DC scanner's output is parsed through the same `_extract_balanced_json` bracket-counting logic Chapter 20B.4 built in full — a greedy regex would happily swallow past a scanner's true closing bracket into trailing prose, exactly the failure mode a sentence-level redundancy scan is prone to producing when the model adds a closing remark after its JSON array.

22B.4.5 Verdict deduplication and out-of-range index guards

Two more defensive checks run before a confirmed group is trusted: the first verdict returned for a given `group_index` wins if the judge somehow emits duplicates, and any `group_index` outside the range of groups actually submitted is discarded as a hallucinated reference rather than trusted.

```
if g_idx < 0 or g_idx >= len(groups):
    logger.warning(f"ignoring out-of-range group_index={g_idx}")
    continue
if g_idx not in verdict_map:   # first verdict wins
    verdict_map[g_idx] = (verdict, reason)
```

Any group the judge never mentions at all defaults to `REJECTED` — the same fail-closed discipline Chapter 20.7's multi-verdict judge chose over the binary judge's fail-open `OK` default, applied here to a per-group decision instead of a whole-answer one.

22B.5 LLM-Based Compression (LBC)

> **Definition — LLM-Based Compression**
> Rewriting a chunk to retain only the sentences that bear on the current query, using an LLM to judge relevance sentence by sentence, guarded by a minimum-retention floor and a faithfulness validator so the rewrite can never fabricate or over-delete.

LBC is the only stage that is query-aware, the most expensive of the three, and the last to run — by the time a chunk reaches LBC, NAC has already restored its continuity and DC has already removed what redundancy could be found without reference to the specific question being asked.

22B.5.1 The LBC compress prompt and the __IRRELEVANT__ sentinel

Each chunk is compressed independently against `_LBC_COMPRESS_PROMPT` (Chapter 14.2), which returns either a trimmed version of the chunk or a literal sentinel string when nothing in the chunk survives the query filter.

```
if compressed_text == "__IRRELEVANT__":
    logger.debug(f"chunk {idx}: marked __IRRELEVANT__ — dropping chunk")
    irrelevant_dropped += 1
    continue
```

A sentinel string, rather than an empty string, is the deliberate choice here — an empty `compressed` field is ambiguous between "nothing is relevant" and "the model produced no output," while `__IRRELEVANT__` can only mean the first.

22B.5.2 LBC_MIN_RETENTION_RATIO — guarding against over-compression

Two structural guards run before any judge call: a retention floor rejects a compression that kept less than `LBC_MIN_RETENTION_RATIO = 0.35` of the original character count, and a symmetric check rejects a "compressed" output that somehow grew *longer* than the original — the exact fabrication pattern Chapter 20B.5 already warned a tolerant repair tool can produce.

```
retention = len(compressed_text) / max(len(original_content), 1)
if retention < min_retention_ratio:
    result.append(chunk); continue          # over-compression guard
if len(compressed_text) > len(original_content):
    result.append(chunk); continue          # over-expansion guard
```

22B.5.3 validate_lbc — detecting fabricated and dropped claims

Only a compression that clears both structural guards reaches `validate_lbc` (Section 22B.6.4) at all — the judge's SAFE / OVER_COMPRESSED / FABRICATED verdict is the last line of defense, not the first, because a judge call is the most expensive check in the stage and the two cheap guards above it catch the two most common failure shapes before spending it.

22B.6 Building validators.py

Four validators, one shared shape: build a prompt from the thing being checked, call `judge_llm`, parse the verdict through `fix_llm_output` (Chapter 20B), and fail closed — `UNKNOWN`, `REJECTED`, or the original unmodified content — on any parse or call failure, never silently accept.

Validator	Verdicts	Guards
`validate_retrieval`	PASS / PARTIAL / FAIL / UNKNOWN	Chapter 12's judge — per-chunk relevance
`validate_merge`	FAITHFUL / UNFAITHFUL / UNKNOWN	Fabricated + dropped claim lists
`validate_redundancy`	CONFIRMED / REJECTED per group	Fail-open call error → all REJECTED (safe default)
`validate_lbc`	SAFE / OVER_COMPRESSED / FABRICATED / UNKNOWN	Fabricated claims + lost relevant facts

22B.6.1 validate_retrieval — PASS / PARTIAL / FAIL per chunk

Every chunk gets an individual `relevant: true/false` verdict with a one-sentence reason, and the overall PASS/PARTIAL/FAIL verdict is meant to summarize them — Chapter 22.7's counting discipline and the top-level-versus-per-chunk consistency risk it implies apply directly here.

22B.6.2 validate_merge — FAITHFUL / UNFAITHFUL with claim lists

The verdict itself is derived, not trusted as emitted: `"UNFAITHFUL" if (fabricated or dropped) else "FAITHFUL"` — the judge's own top-level verdict field is never read at all, only its two claim lists, closing off the exact top-level-disagrees-with-detail risk Section 22B.6.1's validator is still exposed to.

22B.6.3 validate_redundancy — CONFIRMED / REJECTED per group

The only validator in this set that fails open in one narrow, deliberate sense and fails closed in every other: a judge-call error marks every proposed group `REJECTED` rather than `UNKNOWN` — for a stage whose entire job is *removing* content, treating an unresolvable judgment as "don't remove it" is the safe direction to fail in.

22B.6.4 validate_lbc — SAFE / OVER_COMPRESSED / FABRICATED

Three real verdicts plus `UNKNOWN`, and every non-`SAFE` outcome — including `UNKNOWN` — routes to the same place: keep the original chunk. A compression judge only ever gets to make content shorter when it can positively confirm the result is safe, never by default.

22B.7 Extracting compression into its own module

`context_compression.py` exports exactly four public functions — `compress_context_pipeline`, `format_context_for_llm`, `format_precedence_context_for_llm`, and `merge_similar_chunks` — and nothing else needs to reach into its internals. `agent_query.py` never calls NAC, DC, or LBC directly; it calls the pipeline function and trusts the module to sequence its own stages.

This is the same module-boundary discipline Chapter 19B.13 named for `nodes/` versus `services/` — a caller that needs only the pipeline's result should never be able to accidentally couple itself to which internal stage produced it.

22B.8 The compress_context tool

Compression reaches the agent loop as `compress_context`, one of exactly two tools the model can call (Chapter 17.5) — retrieve, then signal that compression should run. Chapter 18.9's message-scrubbing mechanism lives inside this same tool call: once compression succeeds, every earlier `retrieve_documents` result in the message history is replaced with `COMPRESSED_PLACEHOLDER`, because the raw chunks it scrubs are precisely the chunks this chapter's pipeline just condensed.

22B.9 Wiring into agent_query.py and the state machine

Chapter 18.7's COMPRESS phase is where this chapter's pipeline actually runs inside the loop: if the model never called `compress_context` itself, the orchestrator injects the call synthetically (Chapter 18.8.2) before advancing to ANSWER. `agent_state["compress_done"]` is the single flag that tracks whether this has happened yet for the current retrieval round, reset to `False` only when the JUDGE phase sends the loop back to RETRIEVE for another pass.

22B.10 Measuring the token savings

LBC reports its own before-and-after character counts on every run, not as a separate benchmark step but as a normal part of its logging.

```
saved_chars = total_chars_before - total_chars_after
pct = (saved_chars / max(total_chars_before, 1)) * 100
logger.debug(f"chars: {total_chars_before:,} -> {total_chars_after:,} (-
{saved_chars:,} = {pct:.1f}%)")
```

Measured in testing at roughly 27.6% average reduction, LBC's savings compound with whatever NAC and DC already removed upstream — three stages each trimming a smaller, cleaner input than the one before, rather than three independent measurements of the same original context.

22B.11 Known failure modes and tuning knobs

Knob	Value	Controls
`DC_WINDOW_SIZE`	3	Chunks scanned together per redundancy pass
`LBC_MIN_RETENTION_RATIO`	0.35	Floor below which LBC's own output is distrusted
`MERGE_SIMILARITY_THRESHOLD`	0.90	Cosine floor for proposing an intra-retrieval chunk merge

Two real, observed failure modes remain worth naming precisely because their guards are partial, not complete. LBC has fabricated entire paragraphs of plausible-sounding content from citation-only fragments as short as 77 characters — caught only by the length-ratio guard (Section 22B.5.2), a blunt heuristic rather than a semantic check, so a same-length fabrication would pass undetected. DC has permanently deleted a real, query-relevant statistic by wrongly judging it a duplicate of an unrelated, more general sentence — `validate_redundancy` correctly rejected the group afterward, but too late, because DC's string-replace deletion runs before the judge's verdict is even available, and no code path restores content once removed.

Neither failure means the pipeline is unsafe to run — both were caught by the same observability discipline Chapter 22 built, in logs specific enough to show exactly which chunk, which sentence, and which stage. It does mean a compression pipeline this aggressive needs a rollback path its current guards don't yet have: DC's deletion-before-judgment ordering is the more urgent of the two, since fabrication has a length-ratio backstop and destructive deletion currently has none.

Three stages, four validators, and a consistent principle underneath all of them: compression may only ever make content shorter when it can prove the shorter version is still faithful, never as a default outcome of running out of budget or attempts. Chapter 22C takes this exact pipeline and asks what changes when it must run twice at once — once for retrieved documents, once for the self-learning `learned_qa` track — inside the parallel graph Chapter 19B already built.